

OAuth Request Analysis in Burp Suite

Authorization flow review, redirect_uri inspection, state parameter validation, and scope analysis

Field	Details
Module	Sub-Project 2 - API Security Testing Lab
Document Type	Individual API Security Methodology PDF
Phase / Test Case	Phase 10 - OAuth Request Analysis
Prepared For	GitHub Portfolio Documentation
Prepared By	Jagriti Banerjee
Repository	Enterprise-Security-Assessment-Lab
GitHub Filename	10-oauth-request-analysis-methodology-report.pdf

This document is a professional, evidence-driven explanation of one API security methodology section. It is designed for portfolio review and contains only authorized lab-based testing context. Sensitive token values, account data, cookies, and private identifiers are redacted or intentionally represented with placeholders.

1. Section Overview

Area	Details
Phase	Phase 10 - OAuth Request Analysis
Objective	Analyze OAuth request parameters and understand the authorization flow in a controlled lab environment using Burp Suite.
Primary Result	Evidence captured and methodology documented
GitHub Folder	02-api-security/reports/methodology/

Why this section matters

OAuth testing focuses on flow integrity, not just request content. A professional review checks whether redirect_uri is strictly validated, state is present and unpredictable, scopes are minimal, authorization codes are protected, and secrets are not exposed to the browser.

2. Evidence Embedded in This PDF

Evidence 1: 14-oauth-lab-burp-request-analysis.png

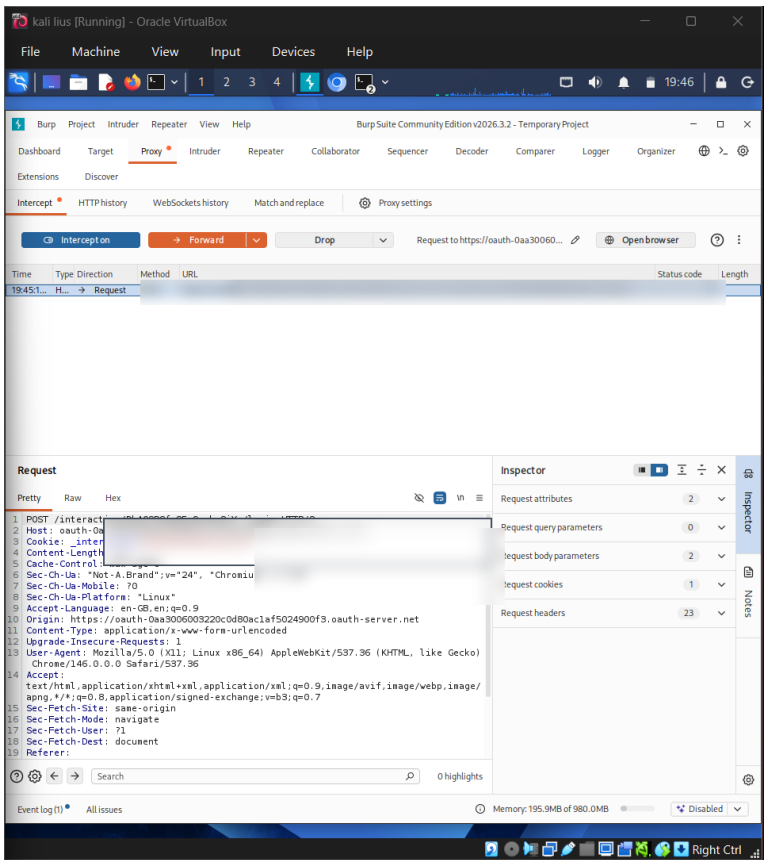


Figure 1 - Burp Suite evidence showing OAuth request review and authorization-flow parameter inspection.

Burp Suite evidence showing OAuth request review and authorization-flow parameter inspection.

3. Command and Workflow Used

```
# OAuth fields reviewed in Burp Suite
client_id
redirect_uri
response_type
scope
state
code
callback URL

# Testing checklist
1. Validate redirect_uri handling
2. Review state value uniqueness and unpredictability
3. Review requested scopes
4. Observe authorization code flow
5. Confirm no client secret exposure in frontend traffic
```

Step-by-step workflow

- Captured OAuth-related browser traffic through Burp Suite.
- Reviewed request parameters such as `client_id`, `redirect_uri`, `response_type`, `scope`, `state`, `code`, and `callback URL`.
- Assessed whether the flow contained obvious misconfiguration indicators.
- Documented request structure and redacted sensitive values before publication.

4. Professional Interpretation

OAuth testing focuses on flow integrity, not just request content. A professional review checks whether `redirect_uri` is strictly validated, `state` is present and unpredictable, scopes are minimal, authorization codes are protected, and secrets are not exposed to the browser.

5. Security Risk and Impact

OAuth misconfigurations can lead to account takeover, token theft, authorization-code interception, open redirect abuse, excessive scope consent, and unauthorized access to third-party resources.

6. Recommended Remediation and Hardening

- Strictly validate `redirect_uri` against an allowlist.
- Use and verify unpredictable state values.
- Use PKCE where appropriate, especially for public clients.
- Limit OAuth scopes to the minimum required.
- Never expose client secrets in frontend code.
- Protect authorization codes and tokens from logs, URLs, and screenshots.

7. Framework Mapping

Framework Area	Mapping
OWASP API	API2: Broken Authentication / OAuth implementation risk
Security Control	OAuth flow validation, redirect_uri validation, state/PKCE
Evidence Result	OAuth request review completed

8. Sanitized Output Style for GitHub

When documenting this evidence publicly, use placeholders instead of live values. The goal is to prove methodology and security understanding without exposing secrets or private data.

```
Authorization: Bearer <REDACTED_TOKEN>
Cookie: <REDACTED_COOKIE>
email: lab-user@example.com
password: REDACTED
object_id: <CONTROLLED_LAB_ID>
redirect_uri: <REDACTED_OR_LAB_CALLBACK>
```

9. GitHub Redaction Checklist

- Bearer tokens and JWT values must be blurred or replaced with <REDACTED_TOKEN>.
- Email addresses, phone numbers, user IDs, and object identifiers should be blurred when they are not required for understanding the evidence.
- Authorization headers, cookies, session values, OAuth codes, refresh tokens, access tokens, and client secrets must never be visible in public GitHub screenshots.
- If a screenshot contains personally identifiable data, crop the view or blur the affected rows before publication.

10. Publication Note

This report is designed for GitHub portfolio use. It describes authorized lab testing only and avoids production claims. The embedded screenshots have been treated as lab evidence and should remain redacted before upload.